

different kernels) or different CPU architectures (you can very well have an ARM virtual machine on a X86 host when you are using full virtualisation) in this kind of virtualisation.

FreeBSD Jails

FreeBSD Jails is one type of container virtualisation, which is supported on the FreeBSD operating system. This is pretty old technology with roots that go back to 1998 (as per Wikipedia). Each jail is an isolated FreeBSD machine with limited resources and privileges. A FreeBSD Jail has its own IP addresses and its own process *namespace*. One jail cannot see or access the other jails or the host's data and/or processes. The only mode of communication between a process running inside a jail and another jail/host is via the network.

IPC (Inter Process Communication) using UNIX sockets is also possible, but it doesn't have the strict restriction of using *namespaces*, and hence it's disabled, by default. The officially recommended tool for managing FreeBSD Jails is *ezjail*, which can be installed on a FreeBSD system as follows:

```
(FreeBSD >= 10) pkg install ezjail
OR
(FreeBSD < 10) pkg_add ezjail
```

If you prefer installing it via ports, then you can find it at sytutils/ezjail.

Open up `/usr/local/etc/ezjail.conf` in your favourite text editor and take a peek at the settings given there. These are *ezjail*'s settings (the utility itself). If you are using ZFS (there's really no reason for not using it), you should turn on the *ezjail_use_zfs_for_jails* parameter in the configuration file and set *ezjail_jailzfs* for the dataset under which all the jails should be created.

In order to start using *ezjail*, you first need to install the *basejail*, which can be done by running the following command:

```
ezjail-admin install -p
```

Or, run the following command (if you need the base OS source code – in most cases, you don't):

```
ezjail-admin install -s
```

You can even specify the FreeBSD version you want for the *basejail* using the `-r` option. By default, it will choose the version that is the same as that of the host.

This command will fetch the FreeBSD base OS archives from the FTP server and set it up.

Next, let's create our first jail, as follows:

```
ezjail-admin create test '100|10.0.0.2/24'
```

ezjail will create the jail using your specified settings (like mine is at `/usr/jails/test`). As I mentioned earlier, the only way to communicate from host to jail and vice versa is via network sockets; the IP address we assigned above, i.e., `10.0.0.2/24`, is the IP address of the jail. We must have an IP address in the same subnet (e.g., `10.0.0.1/24`) on the host for the networking to work.

Note that a jail doesn't need routing tables like traditional virtual machines, and neither can jail manipulate the firewall and routing tables.

If you have extra public IPs, you can assign one of those to the jail and get direct access to the jail from the Internet. For example, if your public interface is *re0* (the name of the interface depends on your NIC and ReX is the name for realtek NICs), then you can assign a public IP to the jail during creation, as follows:

```
ezjail-admin create test 're0|a.b.c.d/cidr'
```

There is one problem with the above set-up though. Inside the jail, all traffic to 127.0.0.1 is redirected to the first IP address allocated to the jail. So, if you assigned just one public IP to it, then all traffic being sent to the loop-back address from inside the jail would appear to come from the public IP. This causes difficulties in firewalling and it's a security risk as well, because you definitely don't want to open up everything to the outside world.

To allocate two IP addresses to a jail during creation, use the following command:

```
ezjail-admin create test '100|10.0.0.2/24,re0|a.b.c.d/cidr'
```

So now, all traffic to 127.0.0.1 inside the jail will be sent to 10.0.0.2. If you already created the jail, you can modify this configuration at `/usr/local/etc/ezjail/<jailname>`. In this case, the jailname is *test*.

IPFW firewall and jails

IPFW is the default firewall in FreeBSD, and was created by the FreeBSD team itself—of course, with inspiration from other firewalls. FreeBSD supports OpenBSD's PF and NetBSD's IPF as well, but I tend to stick to IPFW because it is better integrated with the OS.

NAT

In most cases, you have just one public IP address, but you definitely want to have network access to your jails; otherwise, they're almost useless (well, there are development-related use cases, but this article isn't about that). IPFW has a built-in facility for NATing, which has the same syntax as the user space NAT daemon *natd*. Let's suppose your public IP address is `a.b.c.d`, and configure the NAT rule, as follows:

```
ipfw nat 123 config ip a.b.c.d same_ports unreg_only
```